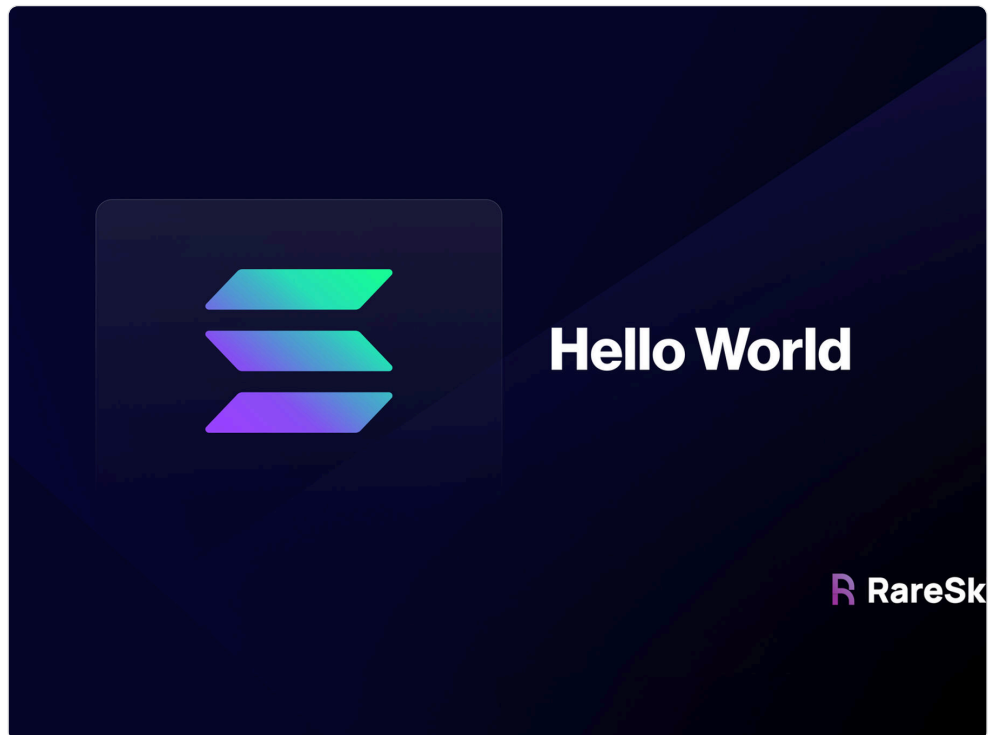


Blog / [Solana Hello World \(Installation and Troubleshooting\)](#)

Solana Hello World (Installation and Troubleshooting)



This is a Solana hello world tutorial. We will walk you through the steps to install Solana and troubleshoot issues that may arise.

If you encounter an issue, please check the Troubleshooting section at the end of this article.

Installation steps

Install Rust

Skip this if you already have Rust installed.

```
# install rust
curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

[Copy](#)

Install Yarn

You will need this to run the unit tests. Skip this if you already have yarn installed.

```
# install yarn -- assumes node js is installed
corepack enable # corepack comes with node js
```

Copy



Apply Now

Install the Solana cli and Anchor

```
# install solana
curl --proto 'https' --tlsv1.2 -sSfL https://solana-
install.solana.workers.dev | bash
```

Copy

Install Anchor

Anchor is a framework for Solana development. It is quite similar to hardhat in many respects.

```
# install anchor
cargo install --git https://github.com/solana-foundation/anchor avm -
-force
avm install latest
avm use latest
```

Copy

Testing the installation

Initialize and build an Anchor Program (for hello world)

Mac users: we recommend calling your program `day_1` instead of `day1` because Anchor seems to silently insert underscores on Mac machines sometimes.

```
anchor init day1 # use day_1 if you have a mac
cd day1
anchor build
```

Copy

Depending on your machine and internet connection, this step may take a while. This is also where you are likely to run into installation problems, so please see the troubleshooting section if necessary.

Configure Solana to run on localhost

```
# shell 1

solana config set --url localhost
```

Copy

Run the test validator node

Run the following command in a new shell, not in the Anchor project. But do not close the shell where you ran `anchor build`. This is running a local (test) Solana node instance on your machine:

```
# shell 2
```

Copy

Ensure the program_id is synced with the Anchor key

Go back to the shell with the Anchor project and run the following command:

```
# shell 1

anchor keys sync
```

Copy

Run the tests

Run this in the Anchor project

```
# shell 1

anchor test --skip-local-validator
```

Copy

The command above runs the test for our program. If you have not already created a test wallet, Anchor will give you instructions for how to do so. We do not provide those instructions here because it will be dependent on your OS and file structure. You may also need to airdrop yourself some local Sol by running `solana airdrop 100 {YOUR_WALLET_ADDRESS}` in the terminal. You can get your wallet address by running `solana address` in the command line.

The expected output is as follows:

```
day_1
Your transaction signature 3bJH3A41cFYdry2JcrAm7QkJcXHEQHE2JY2iwWZcVCixzKgWz5LzotvgXzUzqCje2aFasZ6kqr7RBd6eMoXweFCf
✓ Is initialized! (212ms)

1 passing (214ms)

🚀 Done in 2.97s.
```

Hello World

Now let's make our program output "Hello, world!". Add the following line marked with `****NEW LINE HERE****` to `programs/day_1/src/lib.rs`.

```
use anchor_lang::prelude::*;

declare_id!("...");

#[program]
pub mod day_1 {
    use super::*;

    pub fn initialize(ctx: Context<Initialize>) → Result<()> {
        msg!("Hello, world!"); // **** NEW LINE HERE ****
        Ok(())
    }
}
```

Copy

On This Page

[Installation steps](#)[Testing the installation](#)[Hello World](#)[Realtime Solana logs](#)[Questions and Answers](#)[Troubleshooting](#)[Learn more with RareSkills](#)

```
#[derive(Accounts)]
pub struct Initialize {}
```



Apply Now

Before running the test again, kill the local validator process and restart it with:

```
solana-test-validator --reset
```

Copy

Run the test again

```
anchor test --skip-local-validator
```

Copy

Find the log file by running

```
ls .anchor/program-logs/
```

Copy

Open that file to see the logged "Hello world"

```
(base) ➔ day_1 git:(main) * cat .anchor/program-logs/D9Bks8Miko6tT7XRnM5Kocd3vY6PvSXT6phFunu5ZKjh.day_1.log
Streaming transaction logs mentioning D9Bks8Miko6tT7XRnM5Kocd3vY6PvSXT6phFunu5ZKjh. Confirmed commitment
Transaction executed in slot 1182:
Signature: 2Pv7HbJHTy6KLsjzQ4JZNr7eXSXzKmN1gen4LSvh1UTt6DMwMV4EHrq4jc2n2bXVCrHoYoU9tNFeVQaRQZjdkgNF
Status: Ok
Log Messages:
  Program D9Bks8Miko6tT7XRnM5Kocd3vY6PvSXT6phFunu5ZKjh invoke [1]
  Program log: Instruction: Initialize
  Program log: Hello, world!
  Program D9Bks8Miko6tT7XRnM5Kocd3vY6PvSXT6phFunu5ZKjh consumed 385 of 200000 compute units
  Program D9Bks8Miko6tT7XRnM5Kocd3vY6PvSXT6phFunu5ZKjh success
```

Realtime Solana logs

Alternatively, you can view the logs as they happen by opening a third shell and running:

```
# shell 3

solana logs
```

Copy

Now run the test again, and you should see the same message in the terminal where you ran `solana logs`.

Questions and Answers

Why do `declare_id!` and `msg!` have exclamation points after them?

In Rust, the exclamation point indicates that these are macros. We will revisit macros in a later tutorial.

Do I need an initialize function?

No, that was autogenerated by the Anchor framework. You could name it whatever you like.

There is nothing special about the name `initialize` in this context, and so we can change the name to whatever we like. This is unlike some other keyw languages, like how `main` is a special name in some languages, or how `main` is a special name in Solidity.


[Apply Now](#)

Exercise: Try renaming `initialize` in `programs/day_1/src/lib.rs` and `initialize` in `tests/day_1.ts` to become `initialize2` and run the test again. See the change marked in the **orange circles** below.

```
use anchor_lang::prelude::*;
declare_id!("Fg6PaFpoGXkYsidMpWTK6W2BeZ7FEfcYkg476zPFsLnS");

#[program]
pub mod day_1 {
    use super::*;

    pub fn initialize2(tx: Context<Initialize>) -> Result<> {
        // ...
    }
}

#[derive(Accounts)]
pub struct Initialize {}

import * as anchor from "@project-serum/anchor";
import { Program } from "@project-serum/anchor";
import { Day1 } from "../target/types/day_1";

describe("day_1", () => {
    // Configure the client to use the local cluster.
    anchor.setProvider(anchor.AnchorProvider.env());

    const program = anchor.workspace.Day1 as Program<Day1>;

    it("Is initialized!", async () => {
        // Add your test here.
        const tx = await program.methods.initialize2().rpc();
        console.log("Your transaction signature", tx);
    });
});
```

Why do we run the test with `--skip-local-validator`?

When the tests run against a node, we will be able to query the node for state changes. If you are not able to get the node to run, it is okay to run `anchor test` without the `--skip-local-validator` flag. However, you will have a harder time developing and testing, so we recommend getting the local validator to work.

Troubleshooting

Solana is a rapidly developing software, and you may run into installation issues. We've documented the ones you are most likely to encounter the following sections.

Our tutorial series was written with the following versions:

- Anchor = version 0.29.0
- Solana = version 1.16.25
- Rustc = 1.77.0-nightly

You can change the Anchor version by running:

```
avm install 0.29.0
avm use 0.29.0
```

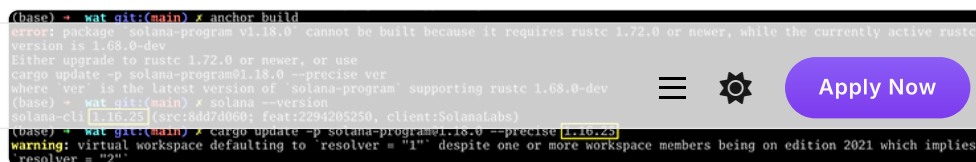
[Copy](#)

error: package `solana-program v1.18.0` cannot be built

```
error: package `solana-program v1.18.0` cannot be built because it
requires rustc 1.72.0 or newer, while the currently active rustc version
is 1.68.0-dev
Either upgrade to rustc 1.72.0 or newer, or use
cargo update -p solana-program@1.18.0 --precise ver
```

[Copy](#)

Check the version of Solana you are running using `solana --version`. Then plug that version into the `ver` above. An example resolution is shown below:



error[E0658]: use of unstable library feature 'build_hasher_simple_hash_one'

If you get the following error:

```
error[E0658]: use of unstable library feature
'build_hasher_simple_hash_one'
  → src/random_state.rs:463:5
  |
463 | / fn hash_one<T: Hash>(&self, x: T) → u64 {
464 | | RandomState::hash_one(self, x)
465 | | }
  | |_____^
  |
= note: see issue #86161 https://github.com/rust-lang/rust/issues/86161
for more information
= help: add #![feature(build_hasher_simple_hash_one)] to the crate
attributes to enable
```

Run the following command: `cargo update -p ahash@0.8.7 --precise 0.8.6`.

Credit: <https://solana.stackexchange.com/questions/8800/cant-build-hello-world>

Error: Deploying program failed: Error processing Instruction 1: custom program error: 0x1

```
Error: Deploying program failed: Error processing Instruction 1: custom
program error: 0x1
There was a problem deploying: Output { status:
ExitStatus(unix_wait_status(256)), stdout: "", stderr: "" }.
```

If you get this error, your keys are not synced. Run `anchor keys sync`.

Error: failed to send transaction: Transaction simulation failed: Attempt to load a program that does not exist

Your keys are not synced. Run `anchor keys sync`.

Error: Your configured rpc port: 8899 is already in use

You ran `anchor test` without `--skip-local-validator` while the validator is running in the background. Either turn off the validator and run `anchor test` or run `anchor test --skip-local-validator` with the validator running. Skip local validator means skip the temporary one it creates for the project, not the one running in the background.

Error: Account J7t...zjK has insufficient funds for spend

Run the command below to airdrop 100 SOL to your development address:

Error: RPC request error: cluster version query failed

```
Error: RPC request error: cluster version query failed: error sending
request for url (http://localhost:8899/): error trying to connect: tcp
connect error: Connection refused (os error 61)
There was a problem deploying: Output { status:
ExitStatus(unix_wait_status(256)), stdout: "", stderr: "" }.
```

Copy

This means the `solana-test-validator` is not running in the background. Run `solana-test-validator` in another shell.

thread 'main' panicked at 'called Option::unwrap() on a None value'

```
thread 'main' panicked at 'called `Option::unwrap()` on a `None` value',
/Users/username/.cargo/git/checkouts/anchor-
50c4b9c8b5e0501f/347c225/lang/syn/src/idl/file.rs:214:73
note: run with `RUST_BACKTRACE=1` environment variable to display
a backtrace
```

Copy

It's likely you didn't run `anchor build` yet.

I'm using Mac and I get Error: failed to start validator: Failed to create ledger at test-ledger: blockstore error

Follow the instructions in this [Stack Exchange thread](#).

I don't have corepack on my Mac despite having node.js

Run the following commands:

```
brew install corepack
brew link --overwrite corepack
```

Copy

Credit: <https://stackoverflow.com/questions/70082424/command-not-found-corepack-when-installing-yarn-on-node-v17-0-1>

error: not a directory:

```
BPF SDK: /Users/rareskills/.local/share/solana/install/releases/stable-
43daa37937907c10099e30af10a5a0b43e2dd2fe/solana-release/bin/sdk/bpf
cargo-build-bpf child: rustup toolchain list -v
cargo-build-bpf child: rustup toolchain link bpf
/Users/rareskills/.local/share/solana/install/releases/stable-
43daa37937907c10099e30af10a5a0b43e2dd2fe/solana-
release/bin/sdk/bpf/dependencies/bpf-tools/rust
error: not a directory:
```

Copy

clear the cache: run `rm -rf ~/.cache/solana/*`.



Error: target/idl/day_1.json doesn't exist. Did you run anchor build?



Apply Now

Create a new project and name it day_1 instead of day1. Anchor seems to silently insert underscores on some machines.

Learn more with RareSkills

This tutorial is the first in our free [Solana Course](#).

Originally Published February, 08, 2024



Looking For An Audit?

Leverage our extensive network of top security specialists.

Get A Quote



YieldNest

Senior Smart Contract Engineer

Join YieldNest as a Senior Smart Contract Engineer to architect and deploy...

Apply Now >

Foreverlong

Backend Engineer

Join us as a Full Stack Developer to architect and ship a...

Apply Now >



MONAD

Senior Front End Engineer

looking for a Senior Frontend Engineer who lives and breathes interface quality...

Apply Now >

Join Our Newsletter

Occasional updates straight to your inbox. No spam.

Your email address

Subscribe



The Web3 education platform trusted by industry-leading developers and auditors.



Blog

Learn Solidity

Events

Testimonials

60 Days of Solana

Contact

Web3 Jobs

ZK Book

Copyright © 2025 RareSkills LLC. All Rights Reserved.

[Privacy Policy](#)

[Terms & Conditions](#)